

L3 MIAGE TL vendredi 11 février 2022

Contrôle continu de cours 30 mn

Répondre de façon concise mais précise aux questions suivantes en utilisant les cadres. Utiliser les notations du cours et du TD. Aucun document n'est autorisé, hormis un aide-mémoire manuscrit de format A4 recto seul constitué par chaque étudiant. Ce sujet comporte 2 pages. Veiller à indiquer vos nom, prénom et numéro étudiant.

1. Répondre aux questions suivantes en justifiant brièvement la réponse.

- Est-ce que la formule $\forall x \in \mathcal{D}. p(x)$ est vraie si $\mathcal{D}$ est vide ?

Oui, parce que $\forall x \in \mathcal{D}. p(x)$ doit se lire $\forall x. x \in \mathcal{D} \Rightarrow p(x)$. Si $\mathcal{D}$ est vide, alors $x \in \mathcal{D}$ est toujours faux, et l'implication est toujours vraie, et la formule aussi.

- Est-ce que la formule $\exists x \in \mathcal{D}. p(x)$ peut être vraie si $\mathcal{D}$ est vide ?

Non, parce que $\exists x \in \mathcal{D}. p(x)$ doit se lire $\exists x. x \in \mathcal{D} \wedge p(x)$. Si $\mathcal{D}$ est vide, alors $x \in \mathcal{D}$ est toujours faux, et la conjonction est toujours fautive, et la formule aussi.

- Soit la formule $\exists n \in \mathbb{N}. \forall x \in [0, 100]. x > n \Rightarrow p(x)$, où $\mathbb{N}$ désigne l'ensemble des entiers naturels. Cette formule est-elle vraie ?

Oui, parce qu'il suffit de choisir $n = 100$. Si $x \in [0, 100]$, alors $x > 100$ est toujours faux, et l'implication toujours vraie, la formule universelle aussi, et donc aussi la formule existentielle.

Ces formules sont importantes, car elles sont à la base de la compréhension de la formule du Lemme de l'étoile.

2. Quels sont les 3 concepts de base de la théorie des langages étudiée en cours ? Expliquer brièvement.

- 1- Les vocabulaires ou alphabets (notés V ou Σ), qui sont des ensembles finis de symboles ou de lettres.
- 2- Les mots, qui sont des séquences finies de lettres ou symboles, éventuellement vides (ε).
- 3- Les langages, qui sont des ensembles de mots, pas forcément finis, même souvent infinis, éventuellement vides ($\emptyset$).

On peut ajouter que les mots sont munis d'une opération de concaténation, notée $\bullet$, associative, non commutative, et dotée d'un élément neutre, ε .

On peut aussi ajouter que les langages sont munis d'une opération de somme, notée $+$, associative, commutative, et dotée d'un élément neutre, $\emptyset$, d'une opération de produit, notée $\bullet$, associative, non commutative, et dotée d'un élément neutre, $\{\varepsilon\}$, et d'une opération de fermeture, dite de Kleene.

Mais l'explication est alors de moins en moins brève !

3. Répondre aux questions suivantes en justifiant brièvement la réponse.

- **Un langage n'est pas rationnel, est-ce qu'un sur-ensemble de ce langage peut être rationnel ?**

Oui, par exemple Σ^* . Plus subtil, prenons $\{ a^n b^n \mid n \geq 0 \}$, $\mathcal{L}$ décrit par $a^* b^*$ en est un sur-ensemble et est rationnel puisqu'on vient de le décrire par une expression régulière.

- **Un langage n'est pas rationnel, est-ce qu'un sous-ensemble de ce langage peut être rationnel ?**

Oui, par exemple $\emptyset$. Plus subtil, prenons $\{ a^n b^n \mid n \geq 0 \}$, $\mathcal{L}$ décrit par $(\emptyset \mid ab \mid aabb \mid aaabbb)$ en est un sous-ensemble et est rationnel puisqu'on vient de le décrire par une expression régulière.

4. Quelles sont les propriétés algébriques de la concaténation ?

La concaténation a un élément neutre, ε , elle est associative, et elle n'est pas commutative.

Ce genre de question montre qu'il ne fallait pas répondre en grand détail à la question 2, puisque l'attention aux détails vient après.

Soit l'alphabet $\Sigma = \{ '-', '.', 's', 'S' \}$. Soit $\mathcal{L}_{\text{morse}}$ le langage des messages codés en morse. Plus précisément, les messages morse sont formés de mots morse séparés par des 'S', les mots morse sont formés de lettres morse séparées par des 's', et les lettres morse sont des séquences non-vides de '-' et de '.'.

5. Proposez une expression régulière qui décrit le langage $\mathcal{L}_{\text{morse}}$.

Il suffit de paraphraser la consigne :

LettreMorse = $('-' \mid '.')^+$

MotMorse = LettreMorse $('s' \text{ LettreMorse})^*$

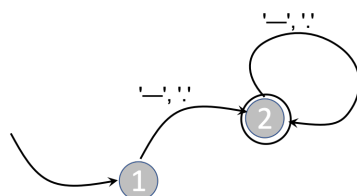
MessageMorse = MotMorse $('S' \text{ MotMorse})^*$

Ce n'est pas comme si je n'avais jamais parlé de trucs séparés par des machins en cours !

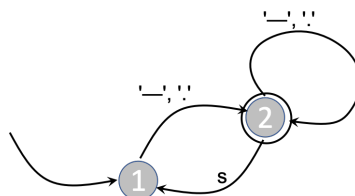
6. Proposez la version graphique d'un automate fini qui reconnaît $\mathcal{L}_{\text{morse}}$.

Construisons-le progressivement :

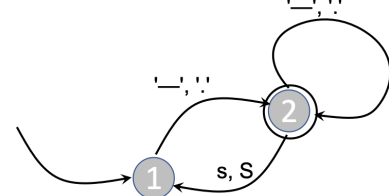
LettreMorse



MotMorse

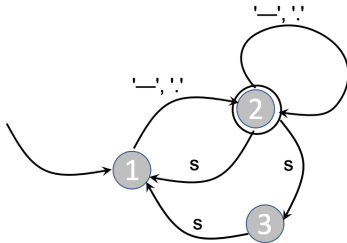


MessageMorse



7. Supposez qu'on décide de séparer les mots morse par des séquences 's•s' plutôt que par des 'S' et qu'on modifie en conséquence votre réponse à la question 6. Est-ce que ce nouvel automate est déterministe ? Si non, est-ce grave ?

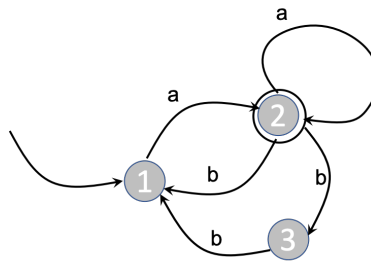
La réponse à la première sous-question dépend bien sûr de votre réponse à la question 6. Dans le cadre de la réponse proposée plus haut,



la réponse est non car en lisant la lettre 's' depuis l'état 2 on peut aller en 1 ou en 3.

Par contre la réponse à la seconde sous-question est « Non, ce n'est pas grave », puisqu'un théorème de Kleene nous dit que si un automate non-déterministe reconnaît un langage, alors un automate déterministe le reconnaît aussi.

8. Soit l'automate $M = \{ \Sigma=\{a, b\}, Q=\{1, 2, 3\}, q_0=1, F=\{2\}, \delta=\{(1,a,2), (2,a,2), (2,b,3), (2,b,1), (3,b,1)\} \}$. Est-il déterministe ? Si non, construisez un automate déterministe qui reconnaît le même langage. Bonus si vous le dessinez.



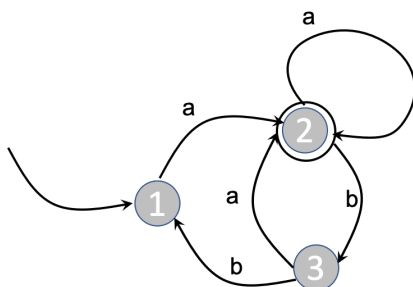
Vision graphique de l'automate M :

l'automate de $\mathcal{L}_{\text{morse}}$ modifié par la question 6 et où on a simplifié le vocabulaire. Il n'est pas déterministe à cause des transitions (2,b,3) et (2,b,1).

En fait, on reconnaît

Il est facile de le déterminer :

Q_{det}	a	b
1	2	
2	2	31
31	2	1



Le bonus :

9. Le langage reconnu par l'automate M est-il rationnel ? Justifier votre réponse.

Oui il est rationnel, parce que le Théorème de Kleene dit que tous les langages rationnels sont reconnus par un automate fini, et que les automates finis ne reconnaissent que des langages rationnels.

10. Proposez un mot de ce langage de longueur supérieure à 5. Illustrez le Lemme de l'étoile en utilisant ce mot.

Soit $m = abaabbaaa$, il peut être découpé en 3 sous-mots, ex. $m = a \bullet baa \bullet bbaaa$, tels que la somme des longueurs des 2 premiers est moindre que 5, et que si on duplique autant de fois qu'on veut le mot du milieu, le résultat est toujours dans le langage, ex. $a \bullet baa \bullet baa \bullet baa \bullet baa \bullet bbaaa$.

Noter que beaucoup de découpages sont possibles, mais que tous ne marchent pas. Par exemple, $a \bullet b \bullet aabbaaa$ ne marche pas, car le langage reconnu par l'automate ne permet pas d'accepter une suite de b plus longue que 2.

L3 MIAGE TL vendredi 11 février 2022

Contrôle continu de TP 30 mn

Répondre de façon concise mais précise aux questions suivantes en utilisant les cadres. Utiliser les notations du TP. Aucun document n'est autorisé, hormis un aide-mémoire manuscrit de format A4 recto seul constitué par chaque étudiant. Ce sujet comporte 2 pages. Veiller à indiquer vos nom, prénom et numéro étudiant.

1. Décrivez comment l'usage du '.' dans la notation des regex diffère de l'usage du '•' dans les notations d'expression régulière du cours et des TD.

Le '•' du cours et des TD représente la concaténation et le produit, mais on avait remarqué qu'on avait l'habitude s'en passer, comme du signe de multiplication en algèbre.

Avec les regex, on décide de s'en passer définitivement, et d'utiliser le point pour dire autre chose. Le '.' des regex signifie un caractère quelconque.

2. Décrivez les grandes lignes de l'usage des '[']' dans la notation des regex.

Les '[']' contiennent des notations de caractères (pas de regex !) et désignent des ensembles de caractères :

- ensemble en extension : [aeiouy] ;
- intervalle : [a-f]
- etc. que vous n'avez pas vraiment utilisé.

3. Décrivez les précautions à prendre quand la notation des regex est plongée dans une chaîne de caractères de Java.

La notation regex utilise des codes spécifiques comme d pour l'ensemble des chiffres, en les préfixant par '\' pour ne pas confondre avec un 'd' ordinaire : d'où '\\d'. À l'inverse, elle utilise certains caractères a priori pour désigner des opérations spécifiques, ex. '*' pour la fermeture de Kleene, et on peut en faire des caractères ordinaires en les préfixant par '\\', d'où '*'.

Tout va bien jusqu'à ce qu'on plonge les regex dans les chaînes de Java, car celles-ci utilisent déjà '\' comme caractère d'échappement, mais pas forcément pour les mêmes échappements. Ex. ni '\\d' ni '*' n'ont de sens pour les chaînes de Java, et le compilateur y verra une anomalie. Il faut donc échapper l'échappement, d'où '\\\\'.

4. Quelle est la notation des regex qui signifie qu'une séquence est optionnelle ? Plusieurs réponses possibles, mais une d'entre elles est vraiment la meilleure. Bonus si vous la mentionnez.

La meilleure : (...) ? .

Une autre : (...) {0,1} , et une autre : (.{0} | ...) .

Soit l'alphabet $\Sigma = \{ '-', '.', 's', 'S' \}$. Soit $\mathcal{L}_{\text{morse}}$ le langage des messages codés en morse. Plus précisément, les messages morse sont formés de mots morse séparés par des 's', les mots morse sont formés de lettres morse séparées par des 's', et les lettres morse sont des séquences non-vides de '-' et de '.'.

5. Proposez une regex qui décrit le langage $\mathcal{L}_{\text{morse}}$. Appelons cette regex morse.

Il suffit de régexifier l'expression régulière produite en réponse à la question 5 du contrôle de cours. On ne peut pas copier la réponse telle quelle parce que la syntaxe n'est pas la même, et on n'a pas de moyen simple de donner des noms à des fragments de regex. On transpose :

```
LettreMorse : (-|\.)+
MotMorse : (-|\.)(s(-|\.))+*
MessageMorse : ((-|\.)(s(-|\.))+*)(s(-|\.)(s(-|\.))+*)*
```

D'où,

```
String morse = "((-|\.)(s(-|\.))+*)(s(-|\.)(s(-|\.))+*)*"
```

6. Quel traitement doit-on appliquer à morse pour tester si une chaîne de caractères est un message morse ?

On doit la compiler (`Pattern p = Pattern.compile(morse)`) puis tester la chaîne avec le pattern obtenu (`boolean b = p.matches(".....")`).

7. Quel traitement doit-on appliquer à morse pour rechercher si une chaîne de caractères contient un message morse ? Si la chaîne contient un message morse, comment connaître sa position ?

On doit la compiler (`Pattern p = Pattern.compile(morse)`), puis créer un matcher (`Matcher m = p.matcher(".....")`), puis interroger itérativement le matcher (`while (m.find()) { ..... }`). Si il répond vrai, on peut connaître la position de l'occurrence qui vient d'être trouvée en la lui demandant (`d = m.start()`).

8. On souhaite préciser que les lettres morse ne peuvent pas être formées de plus de 5 signes '-' ou '.'. Exprimez-le dans une regex. Plusieurs solutions sont possibles : bonus si vous en décrivez une et donnez juste le principe d'une seconde.

La façon "normale" de faire est d'utiliser un quantificateur : `(-|\.){1,5}` .

Une façon très bourrine et pas raisonnable du tout est d'énumérer toutes les longueurs possibles : `(-|\.)|((-|\.)(-|\.))|((-|\.)(-|\.)(-|\.))|((-|\.)(-|\.)(-|\.)(-|\.))|((-|\.)(-|\.)(-|\.)(-|\.)(-|\.))` . Imaginez si la consigne avait été pas plus de 72 signes !

Une autre façon, pas bourrine, mais inutilement compliquée, est d'exprimer qu'on ne veut pas de séquence de plus de 5 signes : $(?!((-|\backslash.)\{6,\}))((-|\backslash.)^*)$.

9. Quel résultat de la théorie des langages est mis en œuvre dans la méthode `compile` ? Que dit ce résultat ?

Ce résultat est le théorème de Kleene (le même que celui de la question 9 de la partie cours). Ce qu'il dit est rappelé dans les éléments de réponse de l'autre question 9. Il répond au questionnement que nous avons eu après avoir présenté les expressions régulières ; elles ont l'air d'être bien utiles pour décrire des langages, mais comment faire pour vérifier automatiquement qu'un mot appartient au langage décrit ? Les automates finis sont la réponse, et le théorème de Kleene nous dit que c'est vraiment une bonne réponse.

10. Soit l'alphabet $\Sigma = \{a, b, c\}$ et s une séquence de Σ^* . On souhaite repérer les occurrences de 'a' dans s qui ont la propriété suivante : « il n'y a pas d'autres 'a' entre ce 'a' et la première occurrence de 'c' qui suit dans s ». Par exemple, dans "babaabcab" seul le troisième 'a' a cette propriété. Expliquez à un stagiaire les grandes lignes de comment s'y prendre.

Façon très cavalière :

Le stagiaire :

« Madame, qu'est-ce que je dois faire ? »

Vous :

« Tu commences par composer une expression régulière ; tu sais ce que c'est ? »

Le stagiaire :

« Oui, j'ai suivi le cours de TL de MIAGE avec M. Ridoux »

Vous :

« Excellent ! Alors vas-y. »

Le stagiaire :

☹

Plus professionnel :

Le stagiaire :

« Madame, qu'est-ce que je dois faire ? »

Vous :

« Tu commences par composer une expression régulière en utilisant la notation des regex ;

Puis tu la compiles en utilisant la méthode `compile` de la classe `Pattern`, ça crée un `pattern` ;

Puis tu crée un `matcher` en utilisant la méthode `matcher` de la classe `Pattern`, ça crée un `matcher` ;

Puis tu l'appelles dans la condition d'une boucle `while` avec la méthode `find` ; Et dans le corps de la boucle, c'est-à-dire quand `find` a rendu vrai, tu attrapes l'occurrence avec les méthodes `start` et `end`.

Tu vois comment ça marche ?

»

Le stagiaire :

« Ah, c'est comme dans le document que vous m'aviez demandé de lire ? »

Vous :

« Tu l'as lu ? Super, tu es prêt. »